

CodeMidas: Scaling Agentic Coding RL Environments from Code Itself

Bowen Ye^{1,2*} Lei Li^{1,3} Shicheng Li¹ Zihao Yue^{1,4} Linghao Zhang¹
 Hanglong Lv^{1,2} Yuanxin Liu¹ Wenhan Ma^{1,2} Hao Tian¹ Rang Li^{1,2}
 Jinhao Dong^{1,4} Yikai Zhao^{1,2} Xiangwei Deng^{1,2} Hailin Zhang¹
 Liang Zhao¹ Qi Liu³ Lingpeng Kong³ Tong Yang^{2,†} Fuli Luo^{1,†}

¹LLM Core, Xiaomi ²Peking University

³University of Hong Kong ⁴Renmin University of China

Abstract

Training capable coding agents via reinforcement learning (RL) requires diverse tasks with reliable verifiers. Open-source codebases offer a rich source of such tasks, while existing methods typically rely on development artifacts such as issues and commits, limiting the range of tasks that can be extracted. To better scale RL environments, we present CodeMidas, an agentic pipeline that turns implemented functionality in existing codebases into executable RL environments using source code as its only task-specific input. CodeMidas allocates agentic compute to every stage of environment construction: agents explore implemented functionality to formulate behavioral specifications, construct tests grounded in execution of the original code, and validate and filter candidate tasks through execution checks and repeated solution rollouts. The resulting dataset has 5,545 training tasks from 3,185 open-source codebases spanning 23 programming languages and 15 technical domains. Training MiMo-V2.5 on these tasks with GRPO improves performance on all five diverse benchmarks, covering issue repair (DeepSWE + 11.7%), whole-program construction (ProgramBench + 17%), and terminal work (Terminal-Bench v2.1 + 8.5%). Ablations show that increasing the number of high-quality training tasks improves performance. Trajectory analysis shows the RL-trained agent demonstrates better behaviors like increasing codebase exploration and more diverse self-verification. These results establish source code as a scalable foundation for constructing RL environments that improve coding agents across diverse software tasks.

1 Introduction

Large language models are increasingly capable of agentic coding: completing substantial pieces of real software work autonomously over long horizons (Deng et al., 2026; Wang et al., 2025; Yang et al., 2024). Building on reinforcement learning (RL) for code generation (Le et al., 2022; Zeng et al., 2025), recent studies have shown substantial gains in real-world software engineering (Chen et al., 2026a; Wei et al., 2025). Effective RL requires diverse tasks and reliable rewards: task diversity supports generalization (Yang et al., 2025), while reliable rewards help reinforce correct work (Badertdinov et al., 2026; Chen et al., 2026a). A central challenge is thus how to turn real codebases into a broad supply of training tasks with trustworthy verifiers.

Existing pipelines construct such environments from development artifacts. Some derive task statements from issues, pull requests, or commits, either directly or via model rewriting (Badertdinov et al., 2026; Chen et al., 2026a; Jain et al., 2025; Jimenez et al., 2024). Others synthesize faults or development tasks around existing tests (Yang et al., 2025; Zeng et al., 2026; Zhang

*Work done during an internship at Xiaomi.

†Co-corresponding authors.

et al., 2025), or use existing documentation to specify the requested functionality (Chen et al., 2026b; Jain et al., 2024). These approaches tie task creation to the coverage of recorded changes, tests, or documentation. This motivates building RL environments directly from code: turning implemented functionality into diverse training tasks at scale, each with reliable verifiers.

Open-source codebases provide a rich foundation for this approach, with large code corpora covering hundreds of programming languages (Lozhkov et al., 2024). Implemented functionality provides both the basis for a task and a candidate solution. Its public interfaces and observable behavior help define what an agent should implement, while executing the original code provides evidence for test expectations. The surrounding codebase can be adapted into a development starting point that preserves real project structure and dependencies. These elements together support the construction of task statements, development environments, and executable verifiers directly from code. Realizing this potential requires making the required behavior explicit in the task statement while leaving internal implementation choices open (Badertdinov et al., 2026). Tests must enforce these requirements, rejecting incorrect solutions while still accepting alternative correct implementations (Barr et al., 2015; Deng et al., 2026; Liu et al., 2023; Wang et al., 2026).

We present **CodeMidas**, an agentic pipeline that automatically constructs executable coding RL environments using source code as its only task-specific input. CodeMidas identifies existing functionality, formulates behavioral task statements, and adapts codebases into development starting points where the target functionality remains to be implemented. It builds tests informed by execution of the original code and checks execution consistency. CodeMidas then applies post-rollout filtering: adversarial rollouts probe for exploitable leakage, solution reviews assess verifier decisions against the stated requirements, and rollout success rates guide task selection.

Using CodeMidas, we construct 5,545 verifiable training tasks from 3,185 open-source codebases spanning 23 programming languages and 15 technical domains. We then train MiMo-V2.5 on these tasks using GRPO (Shao et al., 2024) and observe improvements on all five external benchmarks. Notably, DeepSWE (Huang and Jiang, 2026) pass rate rises from 10.0% to 21.7%, and the ProgramBench (Yang et al., 2026) Almost Solved score rises from 4.5 to 21.5. On Terminal-Bench v2.1 (Merrill et al., 2026), the pass rate rises from 63.7% for the initial policy to 72.2% after RL training. These improvements span issue repair, whole-program construction, code translation, and terminal work, showing that tasks constructed from existing functionality provide strong training signals that transfer across diverse forms of software work.

We further examine how task scale and quality affect these gains, and how agent behavior evolves during training. Increasing the high-quality tasks from 1k to 3k to 5,545 yields progressively higher scores on SWE-bench Pro (Deng et al., 2026), DeepSWE, and CodeMidas Val; even the 3k subset outperforms an 8k baseline constructed without cleaning and filtering on all three. As RL progresses, agents explore codebases more and perform more varied self-verification, with agent-written checks associated with higher success rates. These changes also appear on external tasks, providing behavioral evidence of generalization that complements the benchmark gains.

Together, these findings point to source code itself as a basis for scaling coding RL: implemented functionality can be transformed into verifiable learning environments that support generalization across diverse forms of software work. CodeMidas applies a Midas touch to this resource, turning existing code into RL environments improving coding agents across diverse software tasks.

2 Related Work

2.1 Building coding RL environments

SWE-bench (Jimenez et al., 2024) established repository-level issue resolution as an execution-based evaluation setting. Later pipelines scale task collection from issues and pull requests (Badert-

dinov et al., 2026; Chen et al., 2026a; Fu et al., 2026; Liang et al., 2026; Zhao et al., 2026). R2E-Gym (Jain et al., 2025) generates tests and task statements from commits, reducing reliance on human-written issues and tests. These methods seed tasks from development records.

Other pipelines build tasks around existing tests or documentation. SWE-smith (Yang et al., 2025) synthesizes code changes that break existing tests, while SWE-Flow (Zhang et al., 2025) derives incremental development tasks from unit tests and their runtime dependencies. SWE-Hub (Zeng et al., 2026) combines test-validated bug synthesis with repository construction based on coverage and structured requirements. R2E (Jain et al., 2024) refines function docstrings into specifications, and MindForge (Chen et al., 2026b) exposes documentation and compiled reference programs for from-scratch implementation. Table 1 lists their task-specific input requirements. CodeMidas uses source code as its only task-specific input, deriving behavioral statements and execution-grounded tests beyond the coverage of development records, documentation, and existing tests.

Table 1 Task-specific input requirements and language coverage of representative pipelines for constructing coding environments. ✓ = not required; ♦ = required by part of the pipeline; ✗ = required.

Method	w/o issue	w/o PR	w/o commit	w/o existing tests	w/o written description	Languages
SWE-rebench V2 (Badertdinov et al., 2026)	♦	✗	✗	✗	✓	20
daVinci-Env (Fu et al., 2026)	✗	✗	✗	✗	✓	1
R2E-Gym (Jain et al., 2025)	✓	✓	✗	♦	✓	1
SWE-smith (Yang et al., 2025)	✓	♦	♦	✗	✓	1
SWE-Flow (Zhang et al., 2025)	✓	✓	✓	✗	✓	1
SWE-Hub (Zeng et al., 2026)	✓	✓	✓	✗	♦	11
R2E (Jain et al., 2024)	✓	✓	✓	✓	✗	1
MindForge (Chen et al., 2026b)	✓	✓	✓	✓	✗	15
CodeMidas	✓	✓	✓	✓	✓	23

2.2 Rewards and verification for coding agents

CodeRL (Le et al., 2022) combines unit-test feedback with a learned critic. At repository level, SWE-RL (Wei et al., 2025) uses reference-patch similarity, while SWE-Universe (Chen et al., 2026a) trains agents in executable environments. AceCoder (Zeng et al., 2025) synthesizes tests to study learned rewards and direct test-pass rewards. CodeMidas uses GRPO (Shao et al., 2024) with execution rewards from synthesized tests, without a reward model or learned verifier.

CodeT (Chen et al., 2023) selects programs using generated tests and execution agreement. For SWE agents, SWE-Shepherd (Dihan and Khan, 2026) scores actions with a process reward model, Agentic Rubrics (Raghavendra et al., 2026) scores patches against codebase-grounded rubrics without test execution, and R2E-Gym (Jain et al., 2025) combines learned and execution-based verifiers. Self-Debugging (Chen et al., 2024) and Reflexion (Shinn et al., 2023) use execution feedback or verbal reflection to revise solutions across attempts. Our trajectory analysis examines agents’ exploration and self-verification during RL training and on held-out tasks.

Reliable execution rewards also depend on the test oracle (Barr et al., 2015). EvalPlus (Liu et al., 2023) shows that expanded tests uncover incorrect generated programs missed by original suites, and PatchDiff (Wang et al., 2026) documents incorrect patches accepted by SWE-bench tests. SWE-bench Pro (Deng et al., 2026) and SWE-rebench V2 (Badertdinov et al., 2026) discuss specification gaps and overly restrictive tests. CodeMidas combines execution consistency checks with post-rollout filtering: adversarial rollouts probe leakage, solution reviews check test verdicts, and rollout outcome filtering retains tasks with both successful and failed attempts.

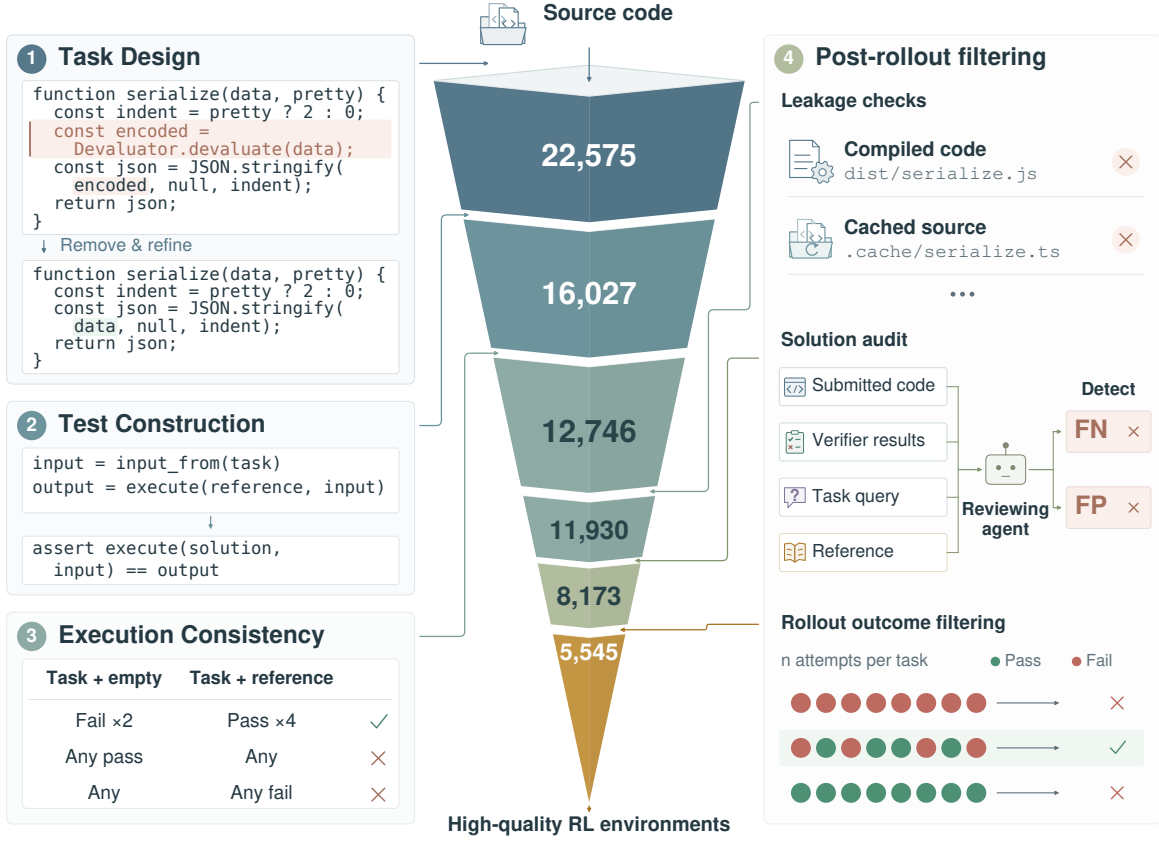


Figure 1 CodeMidas overview. Four modules cover task design, test construction, execution consistency, and post-rollout filtering. The pyramid shows retained task counts respectively.

3 Method

In this section, we describe how CodeMidas constructs and filters coding RL environments using source code as its only task-specific input (Figure 1). We first introduce task design and codebase adaptation (§3.1), followed by execution-grounded test construction (§3.2), and environment preparation with execution consistency check (§3.3). We then describe how agent rollouts are used to filter environments before RL training (§3.4), and summarize the resulting dataset (§3.5).

Each task consists of a statement, a containerized development environment, and a hidden executable verifier. The solver receives the statement and adapted codebase with its dependencies. Throughout solving, the verifier is kept outside the solver’s environment. It is injected only at grading to evaluate the completed implementation and return a binary execution reward for RL.

3.1 Task Design and Codebase Adaptation

An agent inspects codebase structure and build metadata to identify functionality with public entry points and observable outcomes. We prioritize tasks requiring reasoning across the codebase. Supported interfaces include command-line tools, pure library functions, and stateful library APIs, assessed through process outputs, return values, and state changes across calls.

For each candidate, the agent traces public entry points and shared dependencies to define the task scope. It removes the selected core implementation, then adjusts the remaining code to form a coherent starting point for the requested work. The task statement and code boundaries

are revised together while preserving shared components and project context. The original implementation is retained separately to provide a reference solution for the task.

The statement defines inputs, observable behavior, and required public interfaces. Solvers implement the missing codebase functionality, choosing their own internal helpers and algorithms.

3.2 Execution-grounded Test Construction

An agent maps the task statement’s behavioral requirements to test inputs and boundary cases, invokes public entry points in a reference copy of the codebase, and records the outcomes. Tests use command executions for CLI tools, input-output cases for pure functions, and sequences of calls for stateful APIs. Stateful tests exercise dependencies across calls, including ordering and cleanup behavior when specified. Each test records the specific requirement that it covers.

For outputs and properties fixed by the statement, assertions use reference execution to establish expected values. For aspects left unspecified, assertions check only the stated constraints. For example, tests enforce a required exception type without fixing unspecified message wording. Cases with distinct expected outputs probe input-dependent behavior.

An agent then reviews every assertion for restrictions unsupported by the statement, such as exact wording, incidental ordering, or internal structure. It replaces these restrictions with behavioral checks while preserving the checks required by the statement. A task is rejected if an assertion depends on a private symbol and has no behavioral substitute. The revised tests are rerun on the reference solution to confirm that they remain compatible. After review, test inputs and assertions are fixed for grading, which runs the submitted implementation against these checks.

3.3 Environment Preparation

Environment preparation. Starting from a uniform base container image, an agent installs dependencies and prepares build and runtime resources according to the project’s declarations. Cleanup removes artifacts that could reveal the deleted implementation, including compiled outputs, cached copies, and files left by construction agents. Original tests related to the target functionality are also removed. Required packages, fixtures, and build wrappers are retained to support building and running completed implementations in the prepared environment.

Execution consistency. Each task is checked under the training runtime settings in six fresh containers: two with the starting codebase and four with the reference solution in place. Both starting-state runs must fail and all four reference runs must pass. These repetitions check the expected fail-to-pass transition and screen for unstable execution outcomes.

3.4 Post-rollout Environment Filtering

Execution checks cover the starting codebase and the reference solution. Before RL training, we further filter environments using agent rollouts and their outcomes. Post-rollout filtering checks for exploitable leakage, disagreement between solution assessments and test verdicts, and tasks for which all attempts pass or all attempts fail under the screening model.

Leakage filtering. In adversarial rollouts, an agent tries to exploit residual leakage to recover a solution without doing the intended development work. It searches the full solver-visible environment, including compiled artifacts, caches, files left by construction agents, and installed copies of the target project. It logs the commands and outputs supporting each suspected exploit. A separate review checks the evidence against the reference solution and verifier. We reject tasks

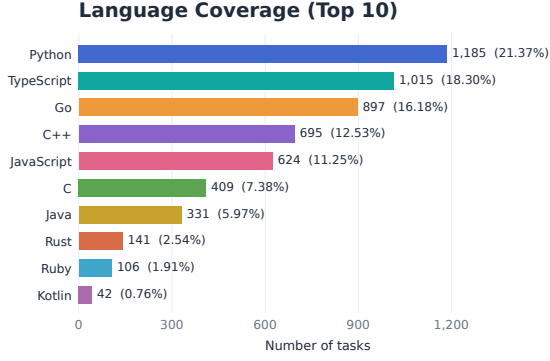


Figure 2 Language coverage of the training set. Tasks inherit the primary language label of their codebase. The ten most frequent languages cover 5,445 of 5,545 tasks (98.2%); the full dataset spans 23 languages. Labels report task counts and the associated percentages of the full training set.

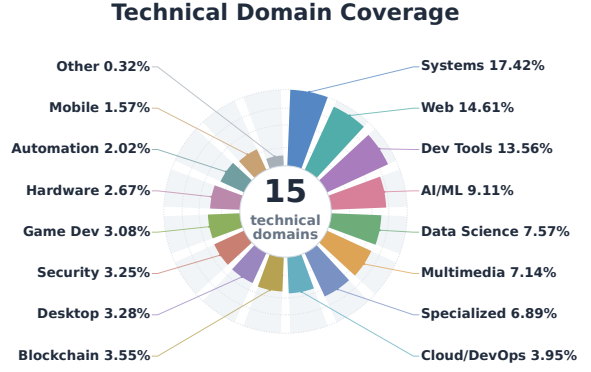


Figure 3 Technical domain coverage of the training set. The 15 labeled domains and Other (18 unlabeled tasks, 0.32%) are ordered clockwise by decreasing task share. Domain labels come from codebases. Radial bar length is proportional to the square root of task share respectively.

if the review confirms that leaked material can bypass the intended implementation work.

Agreement on agent solutions. To assess the verifier on agent-generated implementations, a coding agent tries four times per task. A reviewing agent examines the resulting rollout trajectories, including submitted code and test outputs, alongside the task statement, verifier, and reference solution. Using this evidence, it assesses whether each implementation satisfies the statement and checks for mismatches between the stated requirements and the verifier’s behavior. It flags false positives when an implementation judged incorrect passes the tests, and false negatives when an implementation judged correct fails. Tasks with identified verifier defects are rejected.

Rollout outcome filtering. In another check, a frontier model makes several attempts per task, scored by the verifier. All-pass or all-fail outcomes may reflect task difficulty or remaining defects, such as weak tests or requirements missing from the statement. These outcomes do not reveal the cause. We keep only tasks with both successful and failed attempts under this model and budget.

3.5 Dataset Overview

We retain 5,545 tasks from 3,185 codebases across 23 languages and 15 technical domains.

Language and domain coverage. Figures 2 and 3 summarize the dataset’s coverage across 23

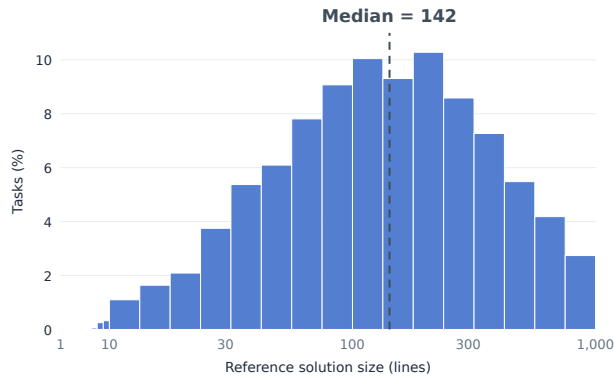


Figure 4 Reference solution size. Bars show task percentages; the dashed line marks the median (142 lines). The piecewise log x-axis compresses the 1–10 interval to one fifth the width of subsequent decades.

programming languages and 15 technical domains. Python (21.4%), TypeScript (18.3%), and Go (16.2%) are the most represented languages, followed by C++ (12.5%) and JavaScript (11.3%). Systems software (17.4%), web technologies (14.6%), and developer tools (13.6%) are the largest technical domains, together accounting for 45.6% of tasks.

Reference solution size. We count all source lines added or deleted in the reference patch, including comments and blank lines. Across the full training set, the median is 142 lines, with an interquartile range of 66–305 lines. Reference patches touch at least two source files in 65.9% of tasks. Figure 4 groups reference solution sizes into equal log-width bins and reports the percentage of all 5,545 training tasks represented in each of these bins.

4 Experiments

In this section, we describe our training and evaluation setup (Section 4.1). We then present results on external benchmarks and examine learning dynamics on CodeMidas Val (Section 4.2).

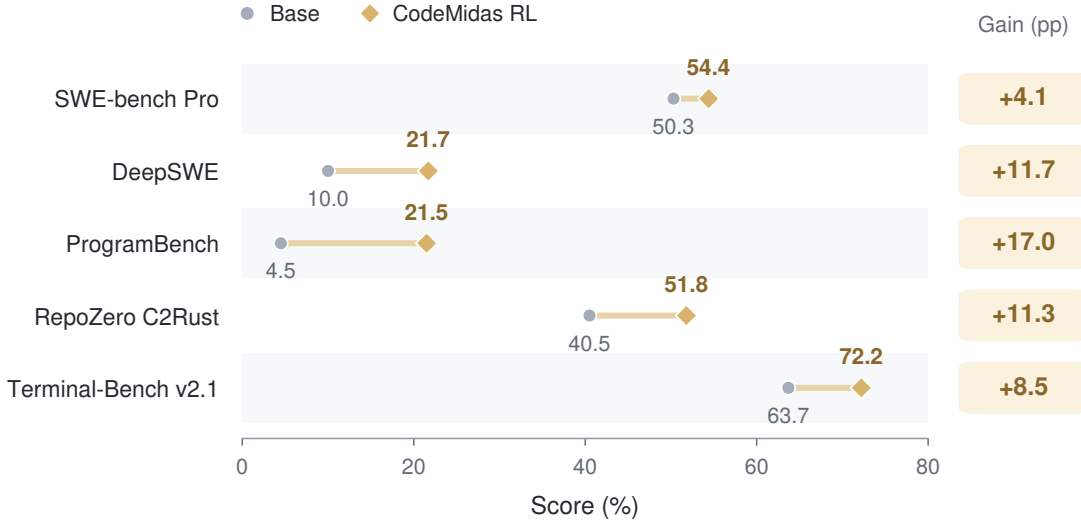


Figure 5 Performance gains from RL on CodeMidas. Initial MiMo-V2.5 scores (gray) and CodeMidas RL scores (gold). ProgramBench reports Almost Solved; all other evaluations report pass rate. Labels on the right give absolute improvements over the initial policy in percentage points.

4.1 Experimental Setup

We train MiMo-V2.5 (Xiaomi MiMo Team, 2026) on 5,545 CodeMidas tasks with GRPO (Shao et al., 2024), binary execution rewards, batch size 32, and 32 rollouts per task. See Appendix A.

We use identical evaluation settings for the initial policy and RL checkpoints. We follow the official task sets for the five external benchmarks: SWE-bench Pro (Deng et al., 2026), DeepSWE v1.1 (Huang and Jiang, 2026), ProgramBench (Yang et al., 2026), RepoZero C2Rust (Zhang et al., 2026), and Terminal-Bench v2.1 (Merrill et al., 2026). CodeMidas Val consists of 200 randomly sampled CodeMidas tasks separate from the 5,545 training tasks, with three evaluation attempts per task. We verified that the training set is disjoint from CodeMidas Val and all five external benchmark task sets. For ProgramBench, we report Almost Solved, the percentage of tasks passing at least 95% of their tests; all other evaluations report pass rate. For each benchmark, we report absolute score improvements relative to the initial policy in percentage points.

4.2 Results

RL improves performance across task types.

Training on CodeMidas improves performance on all five external benchmarks (Figure 5). DeepSWE pass rate increases from 10.0% to 21.7%, while Terminal-Bench v2.1 improves from 63.7% to 72.2%. On Program-Bench, the Almost Solved score rises from 4.5 to 21.5. The gains span repository repair, code translation, program construction, and terminal work, supporting the use of source-derived functionality tasks to train agents for diverse software work.

Learning dynamics on CodeMidas Val. Figure 6 tracks pass rate and mean total token length during RL on CodeMidas. Pass rate rises from 35.0% to 44.7%, staying roughly 8–10 percentage points above the initial rate at evaluated checkpoints from step 40 onward. These gains accompany longer trajectories, indicating greater use of the available interaction budget.

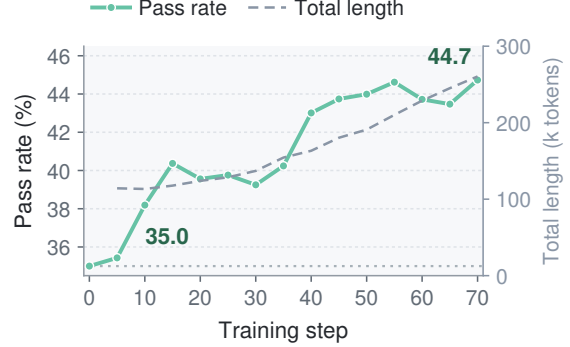


Figure 6 Learning curve on CodeMidas Val. Pass rate (green, left axis) and mean total length (gray-blue dashed, right axis) during RL on CodeMidas. Length is measured in thousands of tokens; the horizontal dotted line marks the pass rate achieved by the initial policy.

5 Analysis

To understand the gains from training on CodeMidas, we examine the contributions of task scale and quality (Section 5.1). We then analyze how agent behavior changes during RL, how these behaviors are associated with task success, and whether they generalize across task types (Section 5.2).

5.1 Task Scale and Quality

To examine the effects of task scale and quality, we train on the full high-quality CodeMidas dataset of 5,545 tasks (5k) and random 1k and 3k subsets. We also train on approximately 8,000 tasks (8k) sampled before filtering, each with a task statement, development environment, and verifier. This vanilla 8k sample is used without environment cleaning or execution consistency checks (Section 3.3), or any of the three post-rollout filtering steps described in Section 3.4.

All four settings use identical training configurations and are evaluated over the same checkpoint range. We first examine learning curves on CodeMidas Val, then compare scores on SWE-bench Pro, DeepSWE, and CodeMidas Val for each of the four training settings.

The full dataset leads across later checkpoints. We first examine every evaluated checkpoint on CodeMidas Val (Figure 7). The 1k setting reaches 41.30 at step 30, while the 3k setting reaches

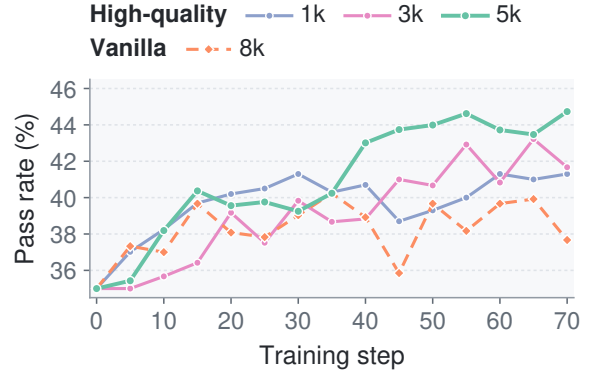


Figure 7 Learning curves on CodeMidas Val. Pass rates at five-step intervals from step 0 to 70, without smoothing. Solid lines with circles denote the high-quality 1k, 3k, and full (5k) pools; the dashed line with diamonds denotes the vanilla 8k sample drawn before filtering and cleaning.

43.22 at step 65. The full dataset leads at every evaluated checkpoint from step 40 through step 70, where it reaches 44.73. Its advantage thus persists across the later part of training.

Performance improves with task scale. We next compare scores across all three evaluations (Figure 8). The 1k, 3k, and 5k pools achieve progressively higher scores: DeepSWE scores rise from 17.57 to 19.05 to 21.70, while scores on CodeMidas Val increase from 41.30 to 43.22 to 44.73. These results support scaling high-quality training data.

The high-quality 5k pool outperforms the vanilla 8k sample. The full CodeMidas dataset exceeds the vanilla 8k sample by 0.59, 4.59, and 4.49 percentage points on SWE-bench Pro, DeepSWE, and CodeMidas Val, respectively. Even the high-quality 3k subset outperforms the vanilla 8k sample on all three evaluations. These comparisons support the combined value of environment reliability and training suitability provided by cleaning, execution checks, and post-rollout filtering, even with fewer tasks and the same training configuration.

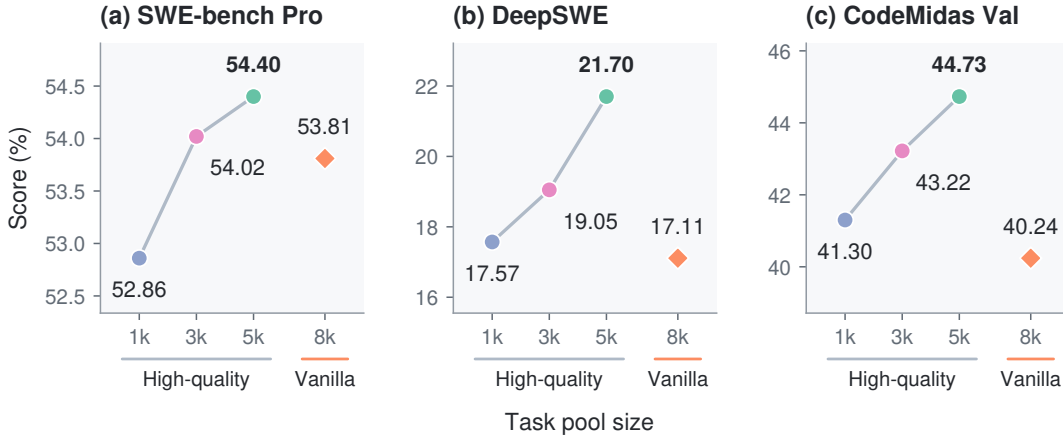


Figure 8 Task scale and quality. Scores on SWE-bench Pro, DeepSWE, and CodeMidas Val. Connected circles denote high-quality 1k, 3k, and 5k pools; diamonds denote vanilla 8k. Panels use different y-axis ranges.

5.2 Behavioral Changes and Generalization

To characterize the behavioral changes accompanying the performance gains, we analyze codebase exploration, reasoning before code edits, and self-verification (Figure 9). For reasoning before edits, we measure the fraction of written code fragments that appeared in preceding reasoning.

Exploration and drafting increase; self-verification diversifies. To quantify behavioral changes during RL, we compare early and late training rollouts (Table 2). We measure exploration by read/search calls before the first edit, drafting by the fraction of code fragments in `Write/Edit` payloads already present in preceding reasoning, and self-verification by distinct verification commands after the final repository edit. Exploration increases from 27.2 to 40.1 calls, the drafting ratio from 0.36 to 0.63, and verification commands from 2.03 to 2.53. Agents thus explore more, show greater overlap between written code and preceding reasoning, and execute more diverse verification commands. Definitions of exploration, code drafting, and self-verification are given in Appendix B.

Self-verification is associated with higher pass rates. Within the same task and checkpoint on CodeMidas Val, rollouts with checks written and executed by the agent have a mean pass rate 4.2 percentage points higher than those without (95% CI: 1.8–6.6). Splitting exploration and drafting at checkpoint medians, excluding undefined measurements, gives differences of +0.7 points (95% CI: -1.9 to 3.7) and +1.95 points (-0.04 to 3.96), respectively. Confidence intervals resample whole tasks, keeping all observed checkpoints from each sampled task together.

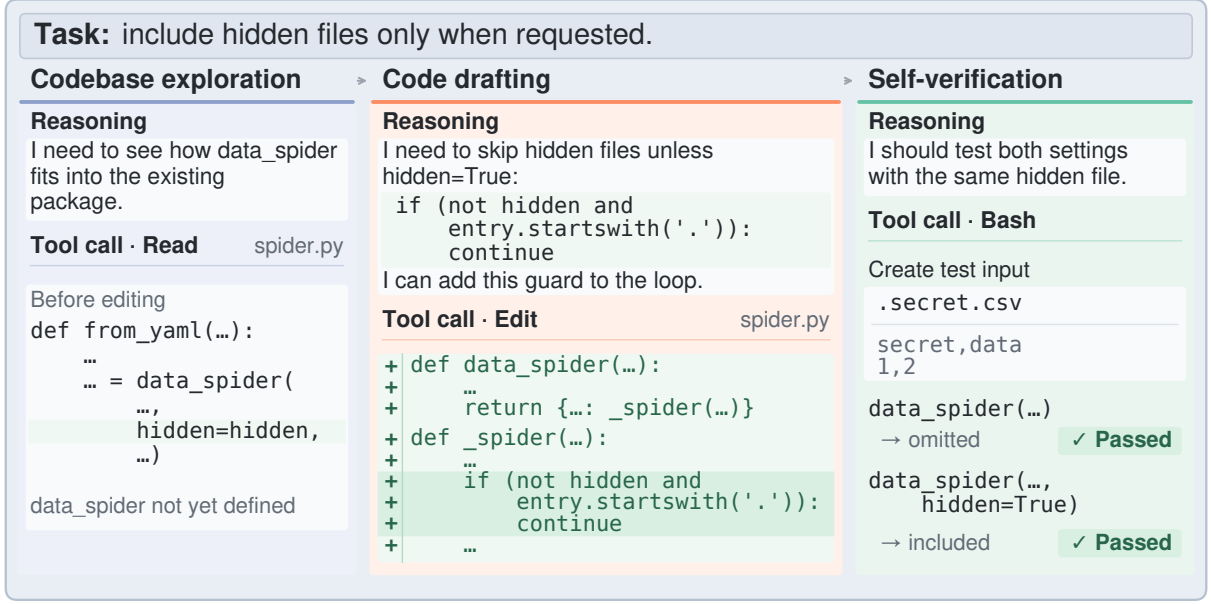


Figure 9 Codebase exploration, code drafting, and self-verification in one rollout. Columns pair reasoning with tool use from one CodeMidas training rollout: reading the caller, drafting hidden-file filtering and applying it via Edit, and testing both flag settings with `.secret.csv`. The diff marks additions; Passed indicates agreement with expected behavior.

Table 2 Changes during RL. Rollout means in early and late training, using defined measurements for each behavior. Changes are computed from unrounded means for each behavioral measure.

Behavior	Measure	Early	Late	Change
Codebase exploration	Pre-edit read/search calls	27.2	40.1	+12.9
Code drafting	Drafting ratio	0.358	0.629	+0.271
Self-verification	Distinct post-edit commands	2.03	2.53	+0.50

Behavioral changes generalize; interaction length varies by task. To assess generalization beyond CodeMidas, we compare early and late checkpoints on SWE-bench Pro, ProgramBench, and Terminal-Bench v2.1 (Table 3). We also measure interaction length in assistant turns to examine how it changes across task types. Exploration increases across all three benchmarks, while changes in verification diversity vary by benchmark. Drafting ratios increase on SWE-bench Pro and ProgramBench. Mean interaction length increases from 37.3 to 50.1 turns for issue repair on SWE-bench Pro and decreases from 155.1 to 122.8 for whole-program construction on ProgramBench. On ProgramBench, greater codebase exploration thus accompanies shorter overall interactions.

Table 3 Behavior on held-out tasks. Means over the first → last three observed checkpoints for each benchmark. The three behavioral measures follow Table 2 and include rollouts with defined measurements; interaction length includes all scored rollouts. Measured subsets can vary across checkpoints.

Evaluation	Codebase exploration (read/search calls)	Code drafting (drafting ratio)	Self-verification (distinct commands)	Interaction length (assistant turns)
SWE-bench Pro	23.1 → 35.5	0.304 → 0.653	0.80 → 0.96	37.3 → 50.1
ProgramBench	55.7 → 83.6	0.106 → 0.361	0.93 → 0.99	155.1 → 122.8
Terminal-Bench v2.1	11.9 → 16.8	N/A	2.01 → 2.61	59.2 → 69.5

6 Conclusion

CodeMidas demonstrates the value of existing codebases as a source of coding RL data. Training on 5,545 tasks improves MiMo-V2.5 across five external benchmarks, with gains spanning diverse forms of software work. Analyses show that expanding high-quality task pools improves performance and that a smaller filtered dataset can outperform a larger unfiltered one. During training, agents explore codebases more and perform more varied self-verification; agent-written checks are associated with higher success rates, and these changes also appear on external tasks.

CodeMidas applies a Midas touch to existing codebases, turning implemented functionality into effective training environments and providing a practical route to scaling coding RL from code itself.

References

- I. Badertdinov, M. Nekrashevich, A. Shevtsov, and A. Golubev. SWE-rebench V2: Language-Agnostic SWE Task Collection at Scale. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=UCAda9kS57>.
- E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015. doi: 10.1109/TSE.2014.2372785. URL <https://doi.org/10.1109/TSE.2014.2372785>.
- B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=ktrw68Cmu9c>.
- M. Chen, L. Zhang, Y. Feng, X. Wang, W. Zhao, R. Cao, J. Yang, J. Chen, M. Li, Z. Ma, H. Ge, Z. Zhang, Z. Cui, D. Liu, J. Zhou, J. Sun, J. Lin, and B. Hui. SWE-Universe: Scale Real-World Verifiable Environments to Millions. arXiv preprint arXiv:2602.02361, 2026a. URL <https://arxiv.org/abs/2602.02361>.
- X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=KuPixIqPiq>.
- Y. Chen, S. Chang, K. Chawa, F. Lin, B. Chen, S. Wang, and A. E. Hassan. MindForge: Teaching Small Language Models Whole-Life-Cycle Software Engineering via Source-Free Program Synthesis. arXiv preprint arXiv:2607.27146, 2026b. URL <https://arxiv.org/abs/2607.27146>.
- X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=uEVTdoAbnK>.
- M. L. Dihan and M. A. R. Khan. SWE-Shepherd: Advancing PRMs for Reinforcing Code Agents. arXiv preprint arXiv:2604.10493, 2026. URL <https://arxiv.org/abs/2604.10493>.
- D. Fu, S. Wu, Y. Wu, Z. Peng, Y. Huang, J. Sun, J. Zeng, M. Jiang, L. Zhang, Y. Li, J. Hu, L. Liu, J. Hou, and P. Liu. daVinci-Env: Open SWE Environment Synthesis at Scale. arXiv preprint arXiv:2603.13023, 2026. URL <https://arxiv.org/abs/2603.13023>.
- W. Huang and P. Jiang. DeepSWE v1.1, 2026. URL <https://deepswe.datacurve.ai/blog/deepswe-v1-1>.
- N. Jain, M. Shetty, T. Zhang, K. Han, K. Sen, and I. Stoica. R2E: Turning any Github Repository into a Programming Agent Environment. In *Proceedings of the 41st International Conference on*

- Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 21196–21224. PMLR, 2024. URL <https://proceedings.mlr.press/v235/jain24c.html>.
- N. Jain, J. Singh, M. Shetty, T. Zhang, L. Zheng, K. Sen, and I. Stoica. R2E-Gym: Procedural Environment Generation and Hybrid Verifiers for Scaling Open-Weights SWE Agents. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=7evvwwdo3z>.
- C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 35, pages 21314–21328. Curran Associates, Inc., 2022. doi: 10.52202/068431-1549. URL <https://doi.org/10.52202/068431-1549>.
- J. Liang, Z. Lyu, Z. Liu, X. Chen, P. Nie, K. Zou, and W. Chen. SWE-Next: Scalable Real-World Software Engineering Tasks for Agents. arXiv preprint arXiv:2603.20691, 2026. URL <https://arxiv.org/abs/2603.20691>.
- J. Liu, C. S. Xia, Y. Wang, and L. Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc., 2023. doi: 10.52202/075280-0943. URL <https://doi.org/10.52202/075280-0943>.
- A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, T. Liu, M. Tian, D. Kocetkov, A. Zucker, Y. Belkada, Z. Wang, Q. Liu, D. Abulkhanov, I. Paul, Z. Li, W.-D. Li, M. Risdal, J. Li, J. Zhu, T. Y. Zhuo, E. Zheltonozhskii, N. O. O. Dade, W. Yu, L. Krauß, N. Jain, Y. Su, X. He, M. Dey, E. Abati, Y. Chai, N. Muennighoff, X. Tang, M. Oblokulov, C. Akiki, M. Marone, C. Mou, M. Mishra, A. Gu, B. Hui, T. Dao, A. Zebaze, O. Dehaene, N. Patry, C. Xu, J. McAuley, H. Hu, T. Scholak, S. Paquet, J. Robinson, C. J. Anderson, N. Chapados, M. Patwary, N. Tajbakhsh, Y. Jernite, C. M. Ferrandis, L. Zhang, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries. StarCoder 2 and The Stack v2: The Next Generation. arXiv preprint arXiv:2402.19173, 2024. URL <https://arxiv.org/abs/2402.19173>.
- M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, D. Lu, O. M. Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V. Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. K. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. K. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, C. M. Rytting, R. Marten, Y. Wang, J. Jitsev, A. Dimakis, A. Konwinski, and L. Schmidt. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- M. Raghavendra, A. Gunjal, B. Liu, and Y. He. Agentic Rubrics as Contextual Verifiers for SWE Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15265–15290. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.697. URL <https://aclanthology.org/2026.acl-long.697/>.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300, 2024. URL <https://arxiv.org/abs/2402.03300>.

- N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc., 2023. doi: 10.52202/075280-0377. URL <https://doi.org/10.52202/075280-0377>.
- X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=0Jd3ayDDoF>.
- Y. Wang, M. Pradel, and Z. Liu. Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, pages 169–181. Association for Computing Machinery, 2026. doi: 10.1145/3744916.3764576. URL <https://doi.org/10.1145/3744916.3764576>.
- Y. Wei, O. Duchenne, J. Copet, Q. Carbonneaux, L. Zhang, D. Fried, G. Synnaeve, R. Singh, and S. I. Wang. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. In *Advances in Neural Information Processing Systems*, volume 38, pages 78500–78525. Curran Associates, Inc., 2025. doi: 10.52202/085713-2629. URL <https://doi.org/10.52202/085713-2629>.
- Xiaomi MiMo Team. MiMo-V2.5, 2026. URL <https://mimo.xiaomi.com/mimo-v2-5>.
- J. Yang, C. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652. Curran Associates, Inc., 2024. doi: 10.52202/079017-1601. URL <https://doi.org/10.52202/079017-1601>.
- J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. SWE-smith: Scaling Data for Software Engineering Agents. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025. doi: 10.52202/085713-3239. URL <https://doi.org/10.52202/085713-3239>.
- J. Yang, K. Lieret, J. Ma, P. Thakkar, D. Pedchenko, S. Sootla, E. McMilin, P. Yin, R. Hou, G. Synnaeve, D. Yang, and O. Press. ProgramBench: Can Language Models Rebuild Programs From Scratch? arXiv preprint arXiv:2605.03546, 2026. URL <https://arxiv.org/abs/2605.03546>.
- H. Zeng, D. Jiang, H. Wang, P. Nie, X. Chen, and W. Chen. ACECODER: Acing Coder RL via Automated Test-Case Synthesis. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12023–12040. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.587. URL <https://aclanthology.org/2025.acl-long.587/>.
- Y. Zeng, S. Li, D. Dong, R. Xu, Z. Chen, L. Zheng, Y. Li, Z. Zhou, H. Zhao, L. Tian, H. Xiao, T. Zhu, L. Hao, and J. Wu. SWE-Hub: A Unified Production System for Scalable, Executable Software Engineering Tasks. arXiv preprint arXiv:2603.00575, 2026. URL <https://arxiv.org/abs/2603.00575>.
- L. Zhang, J. Yang, M. Yang, J. Yang, M. Chen, J. Zhang, Z. Cui, B. Hui, and J. Lin. Synthesizing Software Engineering Data in a Test-Driven Manner. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 76518–76540. PMLR, 2025. URL <https://proceedings.mlr.press/v267/zhang25cn.html>.
- Z. Zhang, Y. Xu, J. Liang, W. Li, X. Chen, L. Qian, X. Pei, J. Huang, R. Sun, and Y. Wu. RepoZero: Can LLMs Generate a Code Repository from Scratch? arXiv preprint arXiv:2605.07122, 2026. URL <https://arxiv.org/abs/2605.07122>.

J. Zhao, G. Chen, F. Meng, M. Li, J. Chen, H. Xu, Y. Sun, W. X. Zhao, R. Song, Y. Zhang, P. Wang, C. Chen, J. Wen, and K. Jia. Immersion in the GitHub Universe: Scaling Coding Agents to Mastery. arXiv preprint arXiv:2602.09892, 2026. URL <https://arxiv.org/abs/2602.09892>.

A Training and Evaluation Configuration

Table A1 lists the evaluation setup and shared training settings for the four task pools in Section 5.1.

Table A1 Training and evaluation configuration.

Setting	Value
Initial policy	MiMo-V2.5
Training task pool	5,545 CodeMidas tasks
Algorithm	GRPO
Reward	Binary verifier outcome (0 or 1)
Advantage normalization by standard deviation	Disabled
Batch size	32
Rollouts per task	32
Maximum prompt length (tokens)	8,192
Maximum response length (tokens)	516,096
Maximum turns per rollout	500
Maximum staleness	8
Optimizer	Adam
Learning rate	5×10^{-6}
Warmup steps	0
Adam (β_1, β_2)	(0.95, 0.95)
Adam ϵ	10^{-15}
Gradient clipping threshold	1
Weight decay	0
CodeMidas Val size	200 tasks
Attempts per task on CodeMidas Val	3

B Behavioral Metric Definitions

Codebase exploration. The number of distinct read/search requests before the first codebase edit. Reads with the same target file and line range, and searches with the same query, scope, and options, are counted once. Equivalent requests through dedicated tools or shell commands are treated as duplicates.

Code drafting. For each Write/Edit payload, we sample distinct 16-character fragments at a stride of four characters. We count fragments found anywhere in reasoning before the corresponding write. The drafting ratio is the summed count divided by the number of fragments sampled across write actions.

Self-verification. The number of distinct verification commands executed after the final codebase edit, including project test commands, inline checks, temporary test programs, and local program runs. Repeated executions of the same verification command are counted only once.